

MATLAB Assignment 2 [Output]

Exercise 1

```
disp("==== Exercise 1 ====");
```

```
==== Exercise 1 ====
```

```
[name, ufid, ...  
 A1, A2, A3, dep_A1, dep_A2, dep_A3, ...  
 B1, B2, B3, dep_B1, dep_B2, dep_B3, ...  
 example_B, dep_B, example_C, dep_C] = Exercise1()
```

```
A1 = 3x5  
    6    7   -1    8    8  
    7    4    3   -2    2  
   -3   -3    8    8    6  
A2 = 3x5  
   -3    6   -4    4    1  
    1    8    7    5    4  
    7    4    8    5   -2  
A3 = 3x5  
    5   -4    5   -4    5  
   -4   -3    0    1    6  
   -1    6    8    0   -2  
dep_A1 =  
'The columns of the matrix forms a linearly DEPENDENT set.'  
dep_A2 =  
'The columns of the matrix forms a linearly DEPENDENT set.'  
dep_A3 =  
'The columns of the matrix forms a linearly DEPENDENT set.'  
dep_B1 =  
'The columns of the matrix forms a linearly INDEPENDENT set.'  
dep_B2 =  
'The columns of the matrix forms a linearly INDEPENDENT set.'  
dep_B3 =  
'The columns of the matrix forms a linearly INDEPENDENT set.'  
example_C = 3x5  
    15    12    9    6    3  
    10     8    6    4    2  
     5     4    3    2    1  
dep_C =  
'The columns of the matrix forms a linearly DEPENDENT set.'  
name =  
"Evan Baesler"  
ufid =  
31151619  
A1 = 3x5  
    6    7   -1    8    8  
    7    4    3   -2    2  
   -3   -3    8    8    6  
A2 = 3x5  
   -3    6   -4    4    1  
    1    8    7    5    4  
    7    4    8    5   -2  
A3 = 3x5  
    5   -4    5   -4    5  
   -4   -3    0    1    6  
   -1    6    8    0   -2  
dep_A1 =  
'The columns of the matrix forms a linearly DEPENDENT set.'  
dep_A2 =  
'The columns of the matrix forms a linearly DEPENDENT set.'
```

```

dep_A3 =
'The columns of the matrix forms a linearly DEPENDENT set.'
B1 = 5×3
    2    -1     2
    1     4     8
    4     4     0
    5    -2     3
    5    -3    -2
B2 = 5×3
    5     8     6
   -1     3    -1
    2    -3     6
    5    -3    -1
    7    -1     8
B3 = 5×3
    0     0    -1
   -2     6     5
   -1     3     5
    4     3     0
    2     7     3
dep_B1 =
'The columns of the matrix forms a linearly INDEPENDENT set.'
dep_B2 =
'The columns of the matrix forms a linearly INDEPENDENT set.'
dep_B3 =
'The columns of the matrix forms a linearly INDEPENDENT set.'
example_B =
NaN
dep_B =
NaN
example_C = 3×5
    15    12     9     6     3
    10     8     6     4     2
     5     4     3     2     1
dep_C =
'The columns of the matrix forms a linearly DEPENDENT set.'

```

```
disp("=====");
```

```
=====
```

Exercise 2

```
disp("==== Exercise 2 ====");
```

```
==== Exercise 2 ====
```

```

[~, ~, ...
 transform_A1, transform_A2, transform_A3, ...
 transform_B1, transform_B2, transform_B3, ...
 C1, C2, C3, transform_C1, transform_C2, transform_C3, ...
 example_neither, transform_neither] = Exercise2(A1, A2, A3, B1, B2, B3)

```

```

check_onto = logical
1
transform_type =
"Onto but not one-to-one"
transform_A1 =
"Onto but not one-to-one"
check_onto = logical
1
transform_type =
"Onto but not one-to-one"

```

```

transform_A2 =
"Onto but not one-to-one"
check_onto = logical
1
transform_type =
"Onto but not one-to-one"
transform_A3 =
"Onto but not one-to-one"
check_onto = logical
0
transform_type =
"One-to-one but not onto"
transform_B1 =
"One-to-one but not onto"
check_onto = logical
0
transform_type =
"One-to-one but not onto"
transform_B2 =
"One-to-one but not onto"
check_onto = logical
0
transform_type =
"One-to-one but not onto"
transform_B3 =
"One-to-one but not onto"
check_onto = logical
1
transform_C1 =
"Onto and one-to-one"
check_onto = logical
1
transform_C2 =
"Onto and one-to-one"
check_onto = logical
1
transform_C3 =
"Onto and one-to-one"
check_onto = logical
0
transform_type =
"Neither onto nor one-to-one"
transform_A1 =
"Onto but not one-to-one"
transform_A2 =
"Onto but not one-to-one"
transform_A3 =
"Onto but not one-to-one"
transform_B1 =
"One-to-one but not onto"
transform_B2 =
"One-to-one but not onto"
transform_B3 =
"One-to-one but not onto"
C1 = 3x3
  -4     6     3
  -4     8     2
   2    -3    -4
C2 = 3x3
   0     0     3
  -2     2    -1
   6    -2     4
C3 = 3x3
   4    -3    -3
   5    -2     6
   1     7     2

```

```

transform_C1 =
"Onto and one-to-one"
transform_C2 =
"Onto and one-to-one"
transform_C3 =
"Onto and one-to-one"
example_neither = 3x3
    1    1    2
    2    3    5
    4    8   12
transform_neither =
"Neither onto nor one-to-one"

```

```
disp("=====");
```

```
=====
```

Exercise 3

```
disp("==== Exercise 3 =====");
```

```
===== Exercise 3 =====
```

```
n = randi([5, 10])
```

```
n =
10
```

```

[~, ~, ...
 A1, A2, A3, A4, ...
 A, B, ABBA, C, AC, CA, AI, IA, inverse_A, ...
 D, inverse_D, rref_something, rref_inverse_D, inv_inv_D, ...
 E, inverse_DE, inv_D_inv_E, inv_E_inv_D, ...
 inv_DT, inv_D_T] = Exercise3(n)

```

Warning: Matrix is close to singular or badly scaled. Results may be inaccurate. RCOND = 1.464556e-19.

```

A1 = 10x10
    2    3    4    5    6    7    8    9   10   11
    3    4    5    6    7    8    9   10   11   12
    4    5    6    7    8    9   10   11   12   13
    5    6    7    8    9   10   11   12   13   14
    6    7    8    9   10   11   12   13   14   15
    7    8    9   10   11   12   13   14   15   16
    8    9   10   11   12   13   14   15   16   17
    9   10   11   12   13   14   15   16   17   18
   10   11   12   13   14   15   16   17   18   19
   11   12   13   14   15   16   17   18   19   20

```

```

A2 = 10x10
    2    3    4    5    6    7    8    9   10   11
    3    4    5    6    7    8    9   10   11   12
    4    5    6    7    8    9   10   11   12   13
    5    6    7    8    9   10   11   12   13   14
    6    7    8    9   10   11   12   13   14   15
    7    8    9   10   11   12   13   14   15   16
    8    9   10   11   12   13   14   15   16   17
    9   10   11   12   13   14   15   16   17   18
   10   11   12   13   14   15   16   17   18   19
   11   12   13   14   15   16   17   18   19   20

```

```

A3 = 10x10
    2    3    4    5    6    7    8    9   10   11
    3    4    5    6    7    8    9   10   11   12
    4    5    6    7    8    9   10   11   12   13

```

5	6	7	8	9	10	11	12	13	14
6	7	8	9	10	11	12	13	14	15
7	8	9	10	11	12	13	14	15	16
8	9	10	11	12	13	14	15	16	17
9	10	11	12	13	14	15	16	17	18
10	11	12	13	14	15	16	17	18	19
11	12	13	14	15	16	17	18	19	20

A4 = 10×10

2	3	4	5	6	7	8	9	10	11
3	4	5	6	7	8	9	10	11	12
4	5	6	7	8	9	10	11	12	13
5	6	7	8	9	10	11	12	13	14
6	7	8	9	10	11	12	13	14	15
7	8	9	10	11	12	13	14	15	16
8	9	10	11	12	13	14	15	16	17
9	10	11	12	13	14	15	16	17	18
10	11	12	13	14	15	16	17	18	19
11	12	13	14	15	16	17	18	19	20

A = 10×10

2	3	4	5	6	7	8	9	10	11
3	4	5	6	7	8	9	10	11	12
4	5	6	7	8	9	10	11	12	13
5	6	7	8	9	10	11	12	13	14
6	7	8	9	10	11	12	13	14	15
7	8	9	10	11	12	13	14	15	16
8	9	10	11	12	13	14	15	16	17
9	10	11	12	13	14	15	16	17	18
10	11	12	13	14	15	16	17	18	19
11	12	13	14	15	16	17	18	19	20

B = 10×8

-3	-1	3	-2	0	3	4	6
1	6	-3	-1	-3	-4	1	-3
-3	1	7	1	6	-1	3	8
8	7	4	-4	1	0	-1	6
-4	-2	0	7	-1	6	5	2
6	-1	2	8	1	-4	-2	1
6	-3	1	2	-3	-4	4	1
7	-3	-4	2	-3	-2	-2	-1
-3	7	-1	0	8	4	0	2
1	3	-3	7	8	5	4	2

ABBA = 10×8

135	88	-12	186	138	43	92	125
151	102	-6	206	152	46	108	149
167	116	0	226	166	49	124	173
183	130	6	246	180	52	140	197
199	144	12	266	194	55	156	221
215	158	18	286	208	58	172	245
231	172	24	306	222	61	188	269
247	186	30	326	236	64	204	293
263	200	36	346	250	67	220	317
279	214	42	366	264	70	236	341

C = 10×10

6	4	-2	-1	1	-1	0	-3	5	6
6	3	1	1	2	1	8	5	-2	3
4	-2	0	3	-3	8	-4	-3	-4	-2
0	-1	8	-1	-1	3	7	4	5	-1
6	2	1	3	6	2	7	2	2	7
2	-2	-2	5	-4	-1	6	6	2	-4
0	6	7	-2	8	2	-3	5	7	2
8	-2	8	-3	5	4	-1	7	3	-2
7	-2	1	-1	2	4	0	7	4	8
3	-2	-3	0	3	1	4	0	7	5

AC = 10×10

271	-10	136	8	161	156	138	244	239	155
313	-6	155	12	180	179	162	274	268	177
355	-2	174	16	199	202	186	304	297	199

```

397    2   193   20   218   225   210   334   326   221
439    6   212   24   237   248   234   364   355   243
481   10   231   28   256   271   258   394   384   265
523   14   250   32   275   294   282   424   413   287
565   18   269   36   294   317   306   454   442   309
607   22   288   40   313   340   330   484   471   331
649   26   307   44   332   363   354   514   500   353
CA = 10x10
  99   114   129   144   159   174   189   204   219   234
 171   199   227   255   283   311   339   367   395   423
 -66  -69  -72  -75  -78  -81  -84  -87  -90  -93
 170   193   216   239   262   285   308   331   354   377
 258   296   334   372   410   448   486   524   562   600
   62    70    78    86    94   102   110   118   126   134
 211   243   275   307   339   371   403   435   467   499
 148   175   202   229   256   283   310   337   364   391
 238   268   298   328   358   388   418   448   478   508
 170   188   206   224   242   260   278   296   314   332
AI = 10x10
   2    3    4    5    6    7    8    9   10   11
   3    4    5    6    7    8    9   10   11   12
   4    5    6    7    8    9   10   11   12   13
   5    6    7    8    9   10   11   12   13   14
   6    7    8    9   10   11   12   13   14   15
   7    8    9   10   11   12   13   14   15   16
   8    9   10   11   12   13   14   15   16   17
   9   10   11   12   13   14   15   16   17   18
  10   11   12   13   14   15   16   17   18   19
  11   12   13   14   15   16   17   18   19   20
IA = 10x10
   2    3    4    5    6    7    8    9   10   11
   3    4    5    6    7    8    9   10   11   12
   4    5    6    7    8    9   10   11   12   13
   5    6    7    8    9   10   11   12   13   14
   6    7    8    9   10   11   12   13   14   15
   7    8    9   10   11   12   13   14   15   16
   8    9   10   11   12   13   14   15   16   17
   9   10   11   12   13   14   15   16   17   18
  10   11   12   13   14   15   16   17   18   19
  11   12   13   14   15   16   17   18   19   20
inverse_A = 10x10
1016 x
-0.2141  -0.0396   0.0208   0.0297   0.0430   0.5125   0.0466  -0.1458 ***
-0.0396   0.0456   0.0108  -0.0490   0.0056   0.0818   0.0131  -0.0744
 0.0208   0.0108  -0.0517  -0.0095   0.0086   0.0727  -0.0025  -0.0809
 0.0297  -0.0490  -0.0095   0.0508  -0.0520   0.0395  -0.0035   0.0131
 0.0430   0.0056   0.0086  -0.0520   0.0426  -0.1479   0.0364   0.0148
 0.5125   0.0818   0.0727   0.0395  -0.1479  -1.7192  -0.2251   0.7830
 0.0466   0.0131  -0.0025  -0.0035   0.0364  -0.2251   0.0331   0.0419
-0.1458  -0.0744  -0.0809   0.0131   0.0148   0.7830   0.0419  -0.2984
-0.1934  -0.0354  -0.0199  -0.0285   0.0605   0.7131   0.0499  -0.3406
-0.0597   0.0415   0.0515   0.0092  -0.0115  -0.1103   0.0102   0.0872
D = 3x3
   1   -1    2
   0    0    1
   1    3   -2
inverse_D = 3x3
 0.7500  -1.0000   0.2500
-0.2500   1.0000   0.2500
   0    1.0000    0
rref_something = 3x6
 1.0000    0    0   0.7500  -1.0000   0.2500
   0    1.0000    0  -0.2500   1.0000   0.2500
   0    0    1.0000    0   1.0000    0
rref_inverse_D = 3x3
 0.7500  -1.0000   0.2500

```

```

    -0.2500    1.0000    0.2500
         0    1.0000         0
inv_inv_D = 3x3
     1    -1     2
     0     0     1
     1     3    -2
E = 3x3
     1     0     2
     2    -1     5
    -1     1    -1
inverse_DE = 3x3
     1.7500    -4.0000     0.2500
     1.2500    -1.5000     0.2500
    -0.5000     1.5000     0.0000
inv_D_inv_E = 3x3
    -0.1250    -0.1250    -1.1250
     0.8750    -0.1250     0.8750
     1.5000    -0.5000     0.5000
inv_E_inv_D = 3x3
     1.7500    -4.0000     0.2500
     1.2500    -1.5000     0.2500
    -0.5000     1.5000         0
inv_DT = 3x3
     0.7500    -0.2500         0
    -1.0000     1.0000     1.0000
     0.2500     0.2500         0
inv_D_T = 3x3
     0.7500    -0.2500         0
    -1.0000     1.0000     1.0000
     0.2500     0.2500         0

```

```
disp("=====");
```

```
=====
```

Extra Credit

```
disp("==== Extra Credit ====");
```

```
==== Extra Credit =====
```

```
[E_matrices_in_order, A_inverse] = ExtraCredit();
```

```
A now is...
NaN
```

```
A now is...
NaN
```

```
Your Inverse Times A is...
NaN
```

```

for i = 1:length(E_matrices_in_order)
    disp("E" + i + " is...")
    disp(E_matrices_in_order{i})
end

```

```

E1 is...
NaN
E2 is...
NaN

```

```
disp("You determined that A^-1 is...")
```

You determined that A^{-1} is...

```
disp(A_inverse)
```

NaN

```
disp("=====");
```

```
=====
```